Lecture 2: Intro to C

Professor G. Sandoval

02

NYU

Some slides adapted by G. Sandoval from slide by Brendan Dolan-Gavitt

C for systems programming

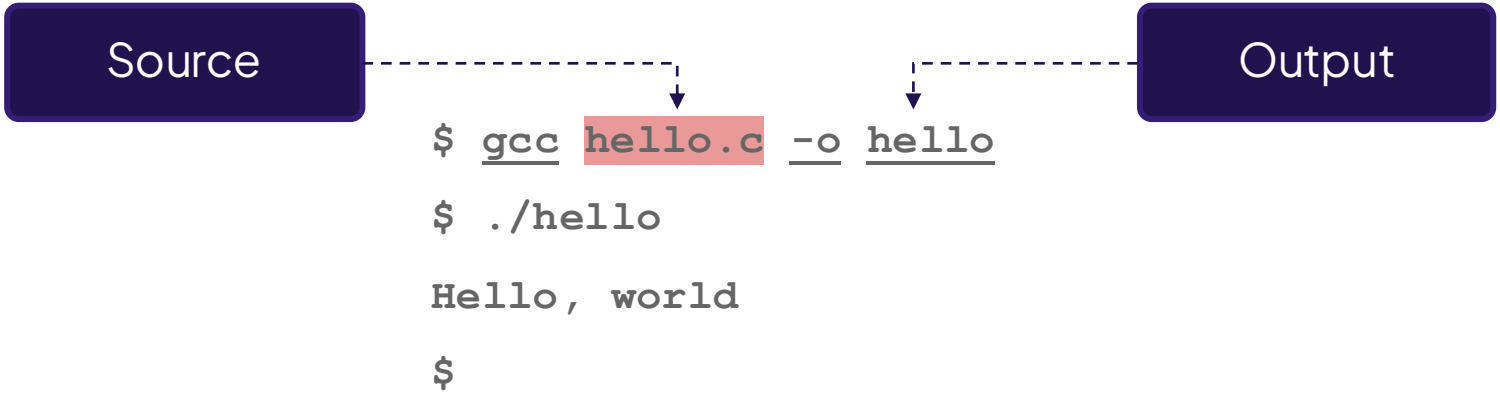
Today we will use C to reason about what the machine does.

Work pattern: predict on your own, compare with a partner, then test.

Focus: compilation, undefined behavior, pointers, raw bytes, ownership, and failure paths.

The syntax reference remains in the original deck.

Compiling and Running



The Compilation Process

Invoking gcc on hello.c did the following things:

a. Called the C ***preprocessor*** (cpp) on hello.c

b. cpp looked for stdio.h in the default include path (a set of directories where header files live) and then recursively called itself on stdio.h

c. The output of that process was pasted into hello.c, and the result was compiled by gcc into an executable binary

Aside: Makefiles

- Remembering and reproducing the commands to compile a project is tedious
- Larger projects may have multiple files, many compilation options, external libraries, etc.
- To manage this complexity we can use a Makefile

NYU

make

- Instead of calling gcc, we can instead just do:

```
$ make hello
cc     hello.c   -o hello
```

- The make command has some built-in rules that understand how to compile basic C and C++ programs
- The make command will check whether the file "hello" exists and is newer than the file "hello.c"; if not, it will try to build "hello" using "hello.c"

Warm-up: where does it fail?

Label each case: compiler, linker, runtime, or correct.

```
1  printf("%d\n", x);  // x was never declared

2  helper();          // declared, never defined

3  int *p = NULL;
   printf("%d", *p);

4  gcc hello.c -o hello;  ./hello
```

NYU

Solo 45 sec. Pair 90 sec. Then vote.

Trace the value of X

```
1   #include <stdio.h>
2   int main() {
3       int x = 0;
4       if (x++ > 0) {
            printf(..)
5           // A will this be printed ?
6       }
7       if (x > 0) {
            printf(. .)
8           // B will this be printed ?
9       }
10      x = 0;
11      if (++x > 0) {
12          printf(..)
13          // C will this be printed ?
14      }
15      if (x > 0) {
          1   Printf(..)
16          // D will this be printed ?
17  }
18      return 0;
19  }
```

Trace x after every condition. Then vote.

NYU

What will be printed?

Only B and D

0%

Only A and C

0%

B, C and D

0%

Nothing

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

More C Operators

- (Pre/Post) Increment/decrement
  - x++: evaluate x, then increment it
  - ++x: increment x, then evaluate it
  - x--: evaluate x, then decrement it
  - --x: decrement, then evaluate it

NYU

Increment example

```c
#include <stdio.h>
int main() {
    int x = 0;
    if (x++ > 0) {
        printf("This will not be printed\n");
    }
    if (x > 0) {
        printf("This will be printed\n");
    }
    x = 0;
    if (++x > 0) {
        printf("This will be printed\n");
    }
    if (x > 0) {
        printf(" This will be printed \n");
    }
    return 0;
}
```

NYU

Arrays

- Arrays in C are *fixed-length* (their size must be specified at compile time)
- To declare an array:
```
int x[10];
```
- Then individual elements are accessible as x[i] where i is an integer

Array Bounds

- Unlike higher-level languages, C **does not check** array bounds
    - In some simple cases, the compiler can detect when you access an out-of-bounds element, but it's not part of the language
- Programming in C requires you to be very careful when programming; there are basically **no safety features**

Predict: an out-of-bounds access

```
1  #include <stdio.h>

2  int main() {
3      int x[5];
4
5      x[10]=9;
6
7      printf("x[10]=%d\n",x[10]);
8      return 0;
9  }
```

NYU

Choose the best answer before the demo.

What happens when we run OOB.c.

Prints 9 with no error

0%

Prints random garbage value

0%

It causes segmentation fault

0%

Undefined behavior. Any of the above is possible

0%

None of the above.

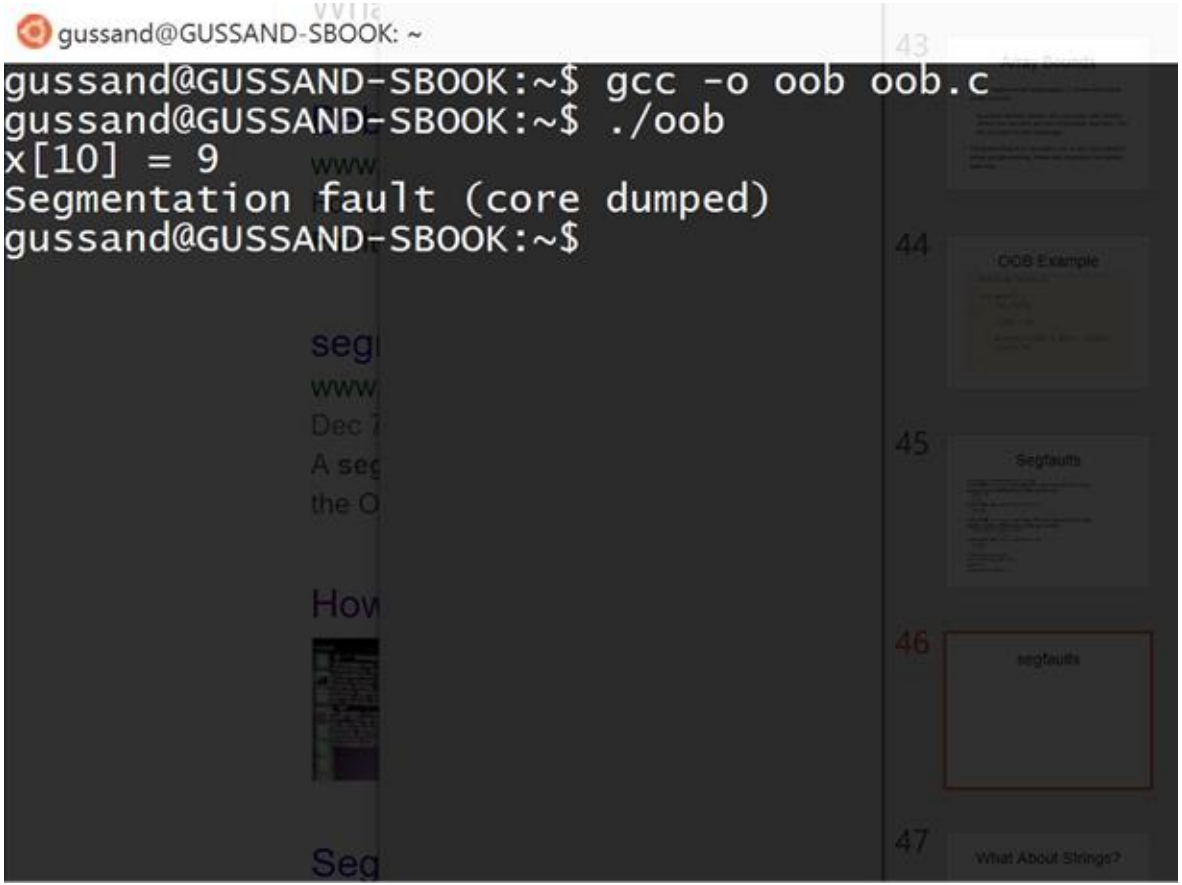
0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Segfaults

```
cosimo:lec2 moyix$ gcc oob.c -o oob
oob.c:6:5: warning: array index 10 is past the end of the array
(which contains 5 elements) [-Warray-bounds]
    x[10] = 9;
    ^ ~~
oob.c:4:5: note: array 'x' declared here
    int x[5];
    ^
oob.c:8:28: warning: array index 10 is past the end of the array
(which contains 5 elements) [-Warray-bounds]
    printf("x[10] = %d\n", x[10]);
                           ^ ~~
oob.c:4:5: note: array 'x' declared here
    int x[5];
    ^
2 warnings generated.
cosimo:lec2 moyix$ ./oob
x[10] = 9
Segmentation fault: 11
```

segfaults


gussand@GUSSAND-SBOOK: ~
gussand@GUSSAND-SBOOK:~$ gcc -o oob oob.c
gussand@GUSSAND-SBOOK:~$ ./oob
x[10] = 9
Segmentation fault (core dumped)
gussand@GUSSAND-SBOOK:~$

What About Strings?

- Strings in C are a **special case of arrays**
- A C string is an **array of characters**, where the **final element** in the array is **the null byte ('\0')**
- So:

```
char hello[] = "hi";
```

- is the same as

```
char hello[] = {'h', 'i', '\0'};
```

NYU

Predict: what does p print?

```
1  #include <stdio.h>

2  int main() {
3      int x=10;
4      int *p = &x;
5
6      printf("%p \n", (void *)p);
7      return 0;
8  }
```

NYU

Sketch x and p before answering.

What will this program print?

The value 10 — 0%

A memory address like (0x16bc) — 0%

NULL — 0%

Compile error — 0%

None of the above — 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Pointers

- C has a special kind of variable called a ***pointer***
- Pointers hold the ***address*** of another variable
- To declare:

```
int *foo;
```

- Two new operators:

  &x : get the address of x

  ***p** : access the value of the pointed-to variable

  (called *dereferencing*)

NYU

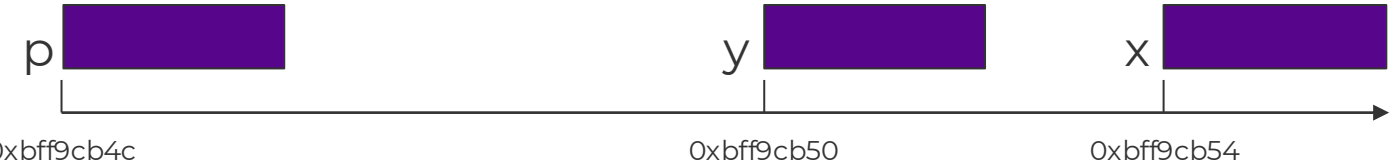
Pointers

```
1   #include <stdio.h>

2   int main() {
3       int *p = NULL;
4       int y = 20;
5       int x = 10;

6       printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7       p = &x;
8       printf("Pointer at %p has value %p and points to var with
9               value %d\n", &p, p, *p);
10      *p = 5;
11      printf("After assigning to *p, x = %d\n", x);
12      p = &y;
13      printf("Pointer at %p has value %p and points to var with
14              value %d\n", &p, p, *p);
15      *p = 50;
16      printf("After assigning to *p, y = %d\n", y);
17      return 0;
18  }
```

Output



Pointers

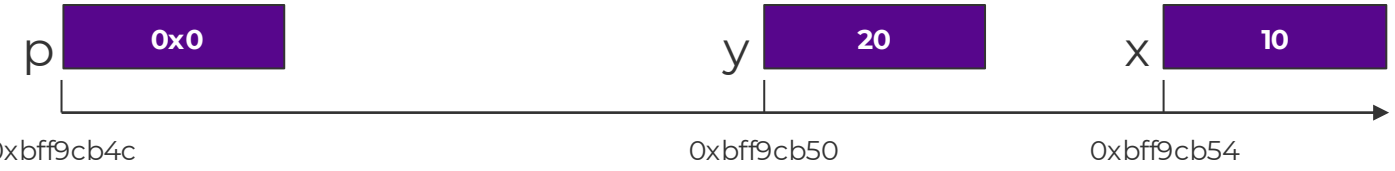
```
1   #include <stdio.h>

2   int main() {
3       int *p = NULL;
4       int y = 20;
5       int x = 10;

6       printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7       p = &x;
8       printf("Pointer at %p has value %p and points to var with
9               value %d\n", &p, p, *p);
10      *p = 5;
11      printf("After assigning to *p, x = %d\n", x);
12      p = &y;
13      printf("Pointer at %p has value %p and points to var with
14              value %d\n", &p, p, *p);
15      *p = 50;
16      printf("After assigning to *p, y = %d\n", y);
17      return 0;
18  }
```

Output

At the start, x = 10, y = 20, p = 0x0



Pointers

```
1   #include <stdio.h>

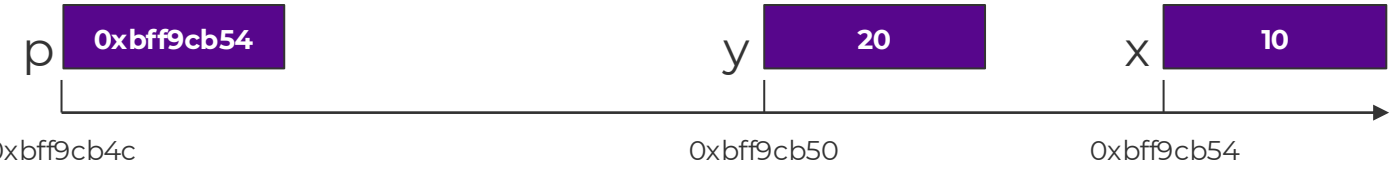
2   int main() {
3       int *p = NULL;
4       int y = 20;
5       int x = 10;

6       printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7       p = &x;
8       printf("Pointer at %p has value %p and points to var with
9               value %d\n", &p, p, *p);
10      *p = 5;
11      printf("After assigning to *p, x = %d\n", x);
12      p = &y;
13      printf("Pointer at %p has value %p and points to var with
14              value %d\n", &p, p, *p);
15      *p = 50;
16      printf("After assigning to *p, y = %d\n", y);
17      return 0;
18  }
```

Output

At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10



Pointers

```
1   #include <stdio.h>

2   int main() {
3       int *p = NULL;
4       int y = 20;
5       int x = 10;

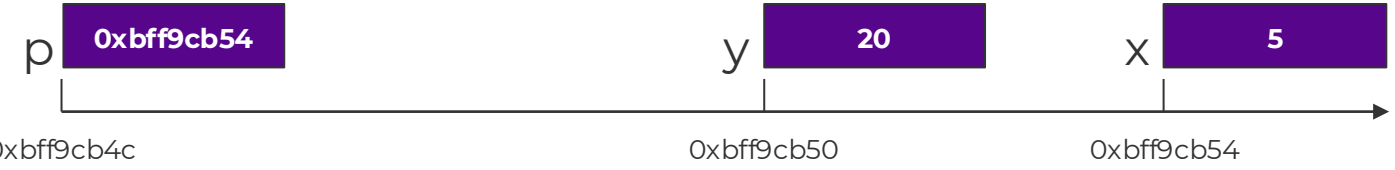
6       printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7       p = &x;
8       printf("Pointer at %p has value %p and points to var with
9               value %d\n", &p, p, *p);
10      *p = 5;
11      printf("After assigning to *p, x = %d\n", x);
12      p = &y;
13      printf("Pointer at %p has value %p and points to var with
14              value %d\n", &p, p, *p);
15      *p = 50;
16      printf("After assigning to *p, y = %d\n", y);
17      return 0;
18  }
```

Output

At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10

After assigning to *p, x = 5



Pointers

```
1   #include <stdio.h>

2   int main() {
3       int *p = NULL;
4       int y = 20;
5       int x = 10;

6       printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7       p = &x;
8       printf("Pointer at %p has value %p and points to var with
9               value %d\n", &p, p, *p);
10      *p = 5;
11      printf("After assigning to *p, x = %d\n", x);
12      p = &y;
13      printf("Pointer at %p has value %p and points to var with
14              value %d\n", &p, p, *p);
15      *p = 50;
16      printf("After assigning to *p, y = %d\n", y);
17      return 0;
18  }
```

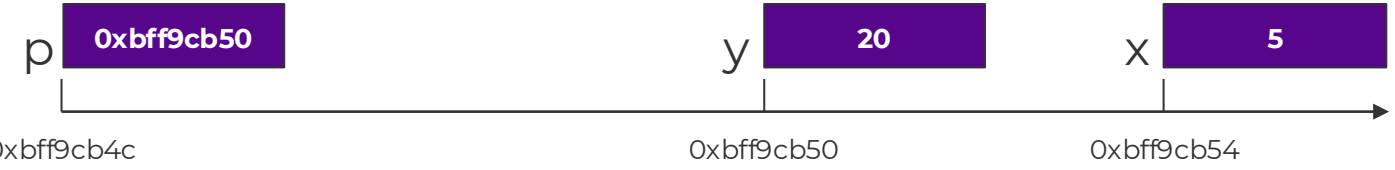
Output

At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10

After assigning to *p, x = 5

Pointer at 0xbff9cb4c
has value 0xbff9cb50 and
points to var with value 20



Pointers

```
1   #include <stdio.h>

2   int main() {
3       int *p = NULL;
4       int y = 20;
5       int x = 10;

6       printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7       p = &x;
8       printf("Pointer at %p has value %p and points to var with
9               value %d\n", &p, p, *p);
10      *p = 5;
11      printf("After assigning to *p, x = %d\n", x);
12      p = &y;
13      printf("Pointer at %p has value %p and points to var with
14              value %d\n", &p, p, *p);
15      *p = 50;
16      printf("After assigning to *p, y = %d\n", y);
17      return 0;
18  }
```

Output

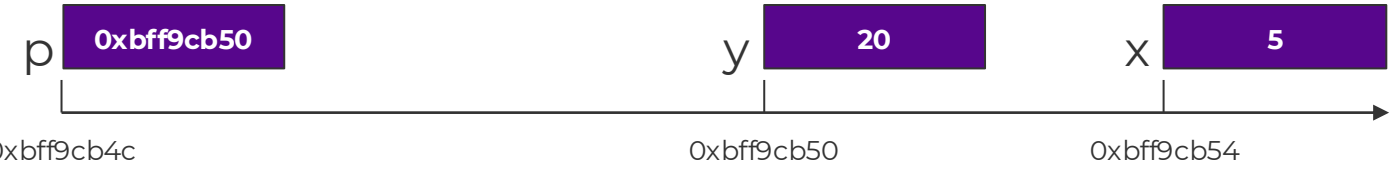
At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10

After assigning to *p, x = 5

Pointer at 0xbff9cb4c
has value 0xbff9cb50 and
points to var with value 20

After assigning to *p, y = 50



Predict the byte order

A 4-byte value is 0x01234567. Its address is 0x100.

Where is byte 0x67 stored on an x86-64 machine?

Little endian stores the least-significant byte at the lowest address.
Big endian stores it at the highest address.
Network byte order is big endian.

Byte Ordering

Example

- Variable x has 4-byte value of 0x01234567
- Address given by &x is 0x100

Big Endian

	0x100	0x101	0x102	0x103		
	01	23	45	67		

Little Endian

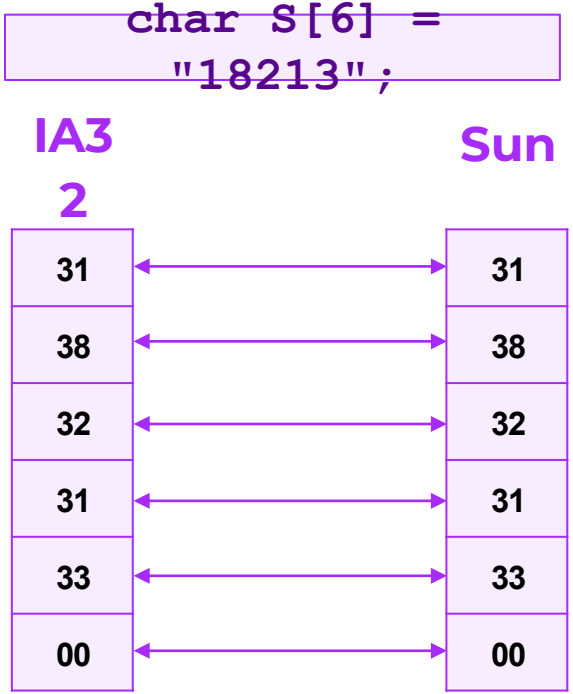
	0x100	0x101	0x102	0x103		
	67	45	23	01		

NYU

Representing Strings

Strings in C

- Represented by array of characters
- Each character encoded in ASCII format
  - Standard 7-bit encoding of character set
  - Character "0" has code 0x30
    - Digit i has code 0x30+i
- String should be null-terminated
  - Final character = 0
- Compatibility
  - **Byte ordering not an issue**



Pair challenge: count newlines in raw bytes

Function Definition:

int **count_newlines(**const char *buf, int n);

Inputs

- buf points to a sequence of **bytes** (often from read).
- n is the number of bytes that are valid to examine.

Behavior

- Return the number of newline characters ('\n') that appear in the **first n bytes** starting at buf.
- Treat the buffer as **raw bytes**, not as a null-terminated string:
- Do **not** stop at '\0'.
- Do **not** call strlen.
- If n <= 0, return 0.

Constraints and edge cases

- **Constraints / assumptions**

- buf is non-NULL when n > 0
- Runs in O(n) time, O(1) extra space.

Example

- buf = "a\nb\nc", n = 5 → returns 2
- buf = {'x','\0','\n'}, n = 3 → returns 1

NYU

Pass-by-value in C

C passes every argument by value.

To modify a caller's object, pass its address.

Changing the local pointer does not change the caller's pointer.

Writing through the pointer changes the pointed-to object.

Pair challenge: swap two integers

Example:

- Write a function called swap that given two integers it swaps it's values:

Call by Value vs Call by Reference

Example:

- Write a function called swap that given two integers it swaps it's values:

```
int x = 42;
int y = 54;
swap (&x, &y);
printf("%d\n", x); // 54
printf("%d\n", y); // 42
```

NYU

Call by Value vs Call by Reference

Example:

- Write a function called swap that given two integers it swaps it's values:

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int x = 42;
int y = 54;
swap (&x, &y);
printf("%d\n", x); // 54
printf("%d\n", y); // 42
```

Structures

- Sometimes we want more complicated data types than just int, char, etc.
- For this we can define a *struct* – an aggregate type that contains several fields

NYU

Data Structures

```c
#include <stdio.h>

struct Person {
    int age;
    int height_cm;
    int weight_kg;
};

int main() {
    int i;
    struct Person p[2];

    p[0].age = 60;
    p[0].height_cm = 150;
    p[0].weight_kg = 90;

    p[1].age = 21;
    p[1].height_cm = 180;
    p[1].weight_kg = 80;

    for (i = 0; i < 2; i++) {
        printf("Person %d is %d years old and weighs %d kg \n",
            i, p[i].age,p[i].weight_kg);
    }
    return 0;
}
```

NYU

One Last Operator

- If you have a pointer to a structure, you can dereference the pointer and access its member in one step:

```
struct Person *p = &q;
p->age = 64;
```

Arrays and Pointers

- Arrays and pointers have a special relationship in C
- An array can be treated as a pointer in most contexts, and vice versa
- If you hand an array of type T to something that expects a pointer to type T, it's treated as if it were a pointer to the first element of the array

Pointer Arithmetic

- You can do arithmetic on pointers
- Given a pointer P of type T, P + N will point to the memory at address P + (N * sizeof(T))
- For example:

```
1  int x [4] = {1, 2, 3, 4};
2  int *p = x;
3  p = p + 2;
4  printf("*p = %d\n", *p);
```

- => Prints **"*p = 3"**

NYU

Pointer Arithmetic vs Array Indexing

- You may have noticed that **x[2]** and ***(p + 2)** refer to the same element
- This is not an accident!
- In fact, we can use array index notation with pointers as well:

```
1  int x [4] = {1, 2, 3, 4};
2  int *p = x;
3  printf("p[2] = %d\n", p[2]);
```

- => Prints **"p[2] = 3"**

NYU

Dynamic memory allocation

Use dynamic allocation when a size or lifetime is known only at runtime.

```
void *p = malloc(number_of_bytes);
```

```
free(p);
```

Check allocation failure and decide who owns the allocation.

Malloc and Free

- **Handle dynamic memory**
- `void*` **malloc** `(size_t size)`:
  - allocate block of memory of `size` bytes
  - does not initialize memory
- `void` **free** `(void* ptr)`:
  - frees memory block, previously allocated by malloc, calloc, realloc, pointed by `ptr`
  - use exactly once for each pointer you allocate
- `size` **argument**:
  - *should* be computed using the **sizeof** operator
  - sizeof: takes a type and gives you its size
  - e.g., `sizeof(int), sizeof(int*)`

NYU

Stack and heap allocation

- Local variables and function arguments are placed on the **stack**
    - deallocated after the variable leaves **scope**
    - **do not return** a pointer to a stack-allocated variable!
    - **do not reference** the address of a variable outside its scope!
- Memory blocks allocated by calls to malloc/calloc are placed on the **heap**
- Globals, constants are placed elsewhere
- Example:
    - // a is a pointer on the stack to a memory block on the heap
    - `int* a = malloc(sizeof(int));`

NYU

Ownership and lifetime rules

- Every successful allocation has one owner.
- Free each allocation exactly once.
- After free, do not dereference the pointer.
- Prefer sizeof *ptr when computing an allocation size.
- Reserve space for the terminating null byte in strings.
- Check allocation results before use.

Example: Linked List

```
1  struct list {
2      int data;
3      struct list *next;
4  };
```

NYU

Pair challenge: insert at the head

```
1  struct list*list_insert(struct list *head, int data) {
2      /* Write code for insert */
3  }
```

Insertion

```
1  struct list * list_insert(struct list *head, int data) {
2      struct list *new = malloc(sizeof(struct list));
3      new->data = data;
4      new->next = head;
5      return new;
6  }
```

Free the list safely

```
Void list_delete(struct list *head) {
/* Write code for freeing all the nodes on the list and */
}
```

NYU

Name the pointer you must save before free.

Example: Linked List

```
1   int main() {
2       int i;
3       struct list *head = NULL;
4       struct list *cursor = NULL;
5       //Add some stuff to our list
6       for (i = 0; i < 20; i++) {
7           head = list_insert(head, i * 2);
8       }
9       //Now print it out
10      cursor = head;
11      while (cursor !=NULL) {
12          printf("list entry at %p has data %d\n",
13                  cursor, cursor->data);
14          cursor = cursor->next;
15      }
16      //And free everything
17      cursor = head;
18      while (cursor != NULL) {
19          struct list *to_delete = cursor;
20          cursor = cursor->next;
21          free(to_delete);
22      }
23  }
```

NYU

Example: Linked List

Why did we have to use the extra to_delete

```
1   int main() {
2       int i;
3       struct list *head = NULL;
4       struct list *cursor = NULL;
5       //Add some stuff to our list
6       for (i = 0; i < 20; i++) {
7           head = list_insert(head, i * 2);
8       }
9       //Now print it out
10      cursor = head;
11      while (cursor !=NULL) {
12          printf("list entry at %p has data %d\n",
13                  cursor, cursor->data);
14          cursor = cursor->next;
15      }
16      //And free everything
17      cursor = head;
18      while (cursor != NULL) {
19          struct list *to_delete = cursor;
20          cursor = cursor->next;
21          free(to_delete);
22      }
23  }
```

Bug hunt: one byte too short

```
1  int main() {
2    char w[strlen("C programming")];
3    strcpy(w, "C programming");
4    printf("%s\n", w);
5    return 0;
6  }
```

NYU

Find the first invalid write. 90 sec.

Vote before the reveal

A The array has exactly enough space

B strcpy truncates the string automatically

C The terminating null byte writes past the array

D The code is valid unless warnings are enabled

Vote A, B, C, or D. Defend your choice.

Answer: one byte too short

```
1  int main() {
2    char w[strlen("C programming")];
3    strcpy(w, "C programming");
4    printf("%s\n", w);
5    return 0;
6  }
```

strlen returns the length of the string not including the null character, so we end up writing a null byte outside the bounds of w.

Bug hunt: a pointer outlives its stack frame

```
1  char *strcdup(int n, char c) {
2  char dup[n+1];
3  int i;
4  for (i = 0; i < n; i++)
5  dup[i] = c;
6  dup[i] = '\0';
7  char *A = dup;
8  return A;
9  }
```

NYU

What becomes invalid at return?

Answer: dangling stack pointer

```
1  char *strcdup(int n, char c) {
2  char dup[n+1];
3  int i;
4  for (i = 0; i < n; i++)
5  dup[i] = c;
6  dup[i] = '\0';
7  char *A = dup;
8  return A;
9  }
```

strcdup returns a stack-allocated pointer. The contents of A will be unpredictable once the function returns.

Capstone: process byte chunks into lines

- **GOAL:** Take a new byte chunk from read() and emit any **complete lines**.
- Input arrives in **chunks of bytes**, not lines
- A line may be split across chunks
- We keep a **carry** buffer for the incomplete last line

Inputs

- **buf, len:** the new bytes (NOT null-terminated; may contain '\0')
- **carry, carry_len:** leftover bytes from the previous call (may be empty)
- **on_line**(**line**, **line_len**): callback to consume each complete line

- **Outputs**
- Calls on_line once per complete line
- Updates carry / carry_len to hold the leftover bytes after processing
-

Teams 8 min. Agree on invariants before code.

Capstone: emit complete lines

Requirements:

Treat input as raw bytes

- Do **not** use strlen, strchr, etc.
- Do **not** stop at '\0'

Combine data:

- Conceptual stream = **old carry + new buf**

Emit lines:

- A "complete line" ends with '\n'
- For each '\n' in the stream, call:

on_line(line_start, line_length)

- line_length includes the trailing '\n'

NYU

Capstone: carry bytes across calls

Update carry:

- After the **last** '\n', any remaining bytes become the **new carry**
- If there are no remaining bytes, carry becomes empty

Memory rule:

- The new carry must **persist** to the next call
- (i.e., store it in allocated memory or a persistent buffer, not in buf)
-

NYU **Carry = bytes after the last newline; emit everything up to and including**

Errors are part of the interface

Before a library call, name its failure signal.

After the call, check that signal before using the result.

Then decide whether the caller can recover or the process must stop.

Examples: malloc and fopen return NULL. strtol reports more detail through endptr and errno.

Class 2 Exit Ticket

0 surveys completed

0 surveys underway

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

When must a buffer be treated as raw bytes instead of a C string?

When it has an explicit length but no guaranteed null terminator

Whenever it contains text

Only after malloc

Only when it is empty

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What must be clear after a successful malloc?

- Which code owns the allocation and must free it
- The exact memory address
- Which compiler allocated it
- Nothing; C manages it automatically

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**